Sistemas Operativos II

Práctica 3 — Informe 1

Carreras críticas en el problema del productor-consumidor con hilos tipo cola

Yago Regueira y Rodrigo Rivademar

Grado en Ingeniería Informática — USC

1. Introducción

En este informe se presenta la adaptación del problema clásico del productor-consumidor al modelo de hilos (pthreads), sustituyendo la pila LIFO de caracteres utilizada en la Práctica 2 por una cola FIFO circular de enteros. El buffer compartido almacena números enteros de dos cifras (1-99) leídos desde un archivo de texto, y tanto el productor como el consumidor calculan la suma de los valores procesados.

El objetivo es demostrar que, sin ningún mecanismo de sincronización, las carreras críticas sobre la variable de control `count` provocan inconsistencias graves.

2. Compilación y ejecución

El programa se compila con gcc:

```
gcc cons_prod.c -o cons_prod
```

Para ejecutarlo:

```
./cons_prod
```

El archivo de texto (archivo.txt) debe contener al menos 80 enteros de dos cifras separados por espacios. Estes 80 enteros se pueden crear manualmente o como hicimos nosotros con:

```
shuf -i 1-99 -n 80 | tr '\n' ' ' > archivo.txt
```

3. Diseño de la solución

3.1. Cambios respecto a la Práctica 2

La estructura de datos pasa de una pila LIFO de caracteres (`stack[N]` + `cima`) a una cola FIFO circular de enteros (`cola[N]` + `inicio` + `fin` + `count`). El campo `inicio` marca la posición de lectura del consumidor, `fin` la de escritura del productor, y `count` lleva la cuenta de elementos almacenados en la propia cola. Ambos índices avanzan de forma circular con la siguiente operacion:

```
buffer.posicion = (buffer.posicion + 1) % N;
```

El tipo de dato cambia de `char` a `int`: la función `produce_item()` lee enteros del archivo con `fscanf("%d")` en vez de caracteres con `fgetc()`, y tanto productor como consumidor guardan la suma de los valores procesados en vez de el recuento de vocales.

3.2. Ausencia de sincronización

El programa no utiliza ningún mecanismo de sincronización: ni semáforos, ni mutex, ni variables de condición. Se realiza una espera activa sobre la variable count: para

controlar el posible desbordamiento del buffer: while(buffer.count == N) para el productor y `while(buffer.count == 0)` para el consumidor.

3.3. La región crítica

La región crítica se encuentra en la actualización de `count`. Para forzar que la carrera sea observable, se parte esta operación en dos pasos separados por un `sleep(1)`:

```
int tmp = buffer.count;
sleep(1);
buffer.count = tmp + 1;
```

El `sleep()` garantiza que el planificador asigne la CPU al otro hilo entre la lectura y la escritura de count.

EJEMPLO DE FLUJO

Si el consumidor lee `count = 3`, duerme, y mientras tanto el productor escribe `count = 4`, al despertar el consumidor escribe `count = 2` (ya que guardo que el valor de `count` era 3). Se pierde el incremento del productor.

4. Resultados

En esta sección se presentan las evidencias obtenidas de la ejecución del programa.

4.1. Corrupción de count

La salida del programa muestra un patrón repetitivo: el productor actualiza `count` a 2, pero inmediatamente después el consumidor lo actualiza a 0, cuando el valor correcto debería ser 1. Esto ocurre en prácticamente cada par de iteraciones, confirmando que la carrera se manifiesta de forma sistemática y no aleatoria con los `sleep()` insertados. Esto se refleja claramente en la Figura 1.

```
[Iteración: 27]Productor: Introduciendo 17 en la posicion 7. Count: 1.
[Iteración: 27]Productor: Count actualizado a 2.
[Iteración: 13]Consumidor: Count actualizado a 0.
```

Figura 1: Primeras iteraciones mostrando la corrupción de count. El productor actualiza count a 2, pero el consumidor lo sobrescribe a 0 al usar un valor obsoleto.

4.2. Bloqueo del programa

El efecto más grave de la corrupción de count es que el programa se bloquea indefinidamente. En la ejecución realizada, el productor completó sus 80 iteraciones, pero el consumidor se detuvo en la iteración 39, habiendo consumido solo 40 de los 80 items. El `count` corrupto alcanzó el valor 0 cuando aún quedaban 40 datos en el buffer. Como el productor ya había terminado, nadie incrementó `count` de nuevo, y el consumidor quedó atrapado en `while(buffer.count == 0)` de forma permanente. Esto se observa claramente en la Figura 2.

Las sumas no llegaron a imprimirse porque el programa nunca terminó.

```
[Iteración: 79]Productor: Introduciendo 91 en la posicion 9. Count: 1.
[Iteración: 79]Productor: Count actualizado a 2.
[Iteración: 39]Consumidor: Count actualizado a 0.
```

Figura 2: Últimas líneas de la ejecución. El productor completa la iteración 79, pero el consumidor se queda bloqueado tras la iteración 39 con count = 0.

4.3. Análisis de la traza

A pesar de que `count` se corrompe, los datos del buffer (los enteros) se transfieren correctamente mientras el programa sigue ejecutándose. Esto se debe a que `inicio` y `fin` avanzan correctamente ya que solo el consumidor utiliza `inicio` y solo el productor `fin`. Para comprobar esto podemos observar las primeras ejecuciones los números coinciden con los del archivo como se refleja en la Figura 3.

```
[Iteración:  0]Productor: Introduciendo 63 en la posicion 0. Count: 0.
[Iteración:  0]Productor: Count actualizado a 1.
[Iteración:  0]Consumidor: retiro 63 de la posicion 0. Count era: 1.
[Iteración:  1]Productor: Introduciendo 94 en la posicion 1. Count: 1.
[Iteración:  1]Productor: Count actualizado a 2.
[Iteración:  0]Consumidor: Count actualizado a 0.
[Iteración:  2]Productor: Introduciendo 50 en la posicion 2. Count: 0.
[Iteración:  2]Productor: Count actualizado a 1.
[Iteración:  1]Consumidor: retiro 94 de la posicion 1. Count era: 1.
```

```
1   63 94 50 90 11
```

Figura 3: Comparación mostrando que los datos se transfieren correctamente

5. Conclusiones

Este ejercicio demuestra que cuando dos hilos acceden a una variable compartida (`count`) sin exclusión mutua, el resultado no es simplemente un valor incorrecto, puede suceder una inconsistencia más grave como el bloqueo total de un programa por la aparición de un interbloqueo.

La solución a este problema se abordará en el Informe 2 mediante el uso de mutex y variables de condición.